

Universidad Simón Bolívar

Departamento de Computación y Tecnología de la Información

CI-3725. Traductores e Interpretes



Etapa II

Análisis Sintáctico

Estudiantes:

Nazareth Colmenares. 20-10017

Gaston García. 19-10668

Mauricio Teixeira. 20-10072

Profesor:

Ricardo Monascal

Etapla II: Análisis Sintáctico

La segunda etapa del proyecto consiste en la implementación del módulo de análisis sintáctico para el lenguaje BOT. A diferencia de la etapa anterior, en esta entrega se busca verificar que una secuencia de tokens forme una estructura sintáctica válida de acuerdo con las reglas del lenguaje, así como construir una representación interna de dicha estructura mediante un Árbol Sintáctico Abstracto (AST).

El objetivo principal del analizador sintáctico es determinar si un programa escrito en BOT pertenece o no al lenguaje definido por la gramática propuesta. En caso de que el programa sea correcto, se construye un árbol sintáctico abstracto que representa las instrucciones del programa y posteriormente se imprime una representación legible de dicho árbol. Si existe algún error sintáctico, únicamente se reporta el primer error encontrado, mientras que los errores léxicos continúan siendo reportados en su totalidad, tal como se estableció en el proyecto.

La implementación fue realizada en Python utilizando la biblioteca PLY (Python Lex-Yacc), y aprovechando los componentes desarrollados en la primera etapa para el análisis léxico y extendiéndolos con un analizador sintáctico generado mediante Yacc.

Inicialmente se realiza un análisis léxico completo del archivo fuente. Si se detecta al menos un error léxico, la ejecución se detiene después de mostrar todos los errores encontrados.

Si el análisis léxico es exitoso, se inicia el análisis sintáctico utilizando la gramática definida en PLY. Durante esta fase se construyen los nodos que conforman el árbol sintáctico abstracto.

Finalmente, si no se detectan errores sintácticos, el árbol es recorrido e impreso en formato legible.

Diseño de la Gramática

Uno de los aspectos más importantes de esta etapa del proyecto fue el diseño de una gramática capaz de reconocer todos los programas válidos escritos en BOT.

Las producciones de la gramática son las siguientes:

$BOT \rightarrow CREATE \ EXECUTE$

$CREATE \rightarrow TkCreate \ DECLARATIONS$

$| \text{ empty}$

$DECLARATIONS \rightarrow DECLARATIONS \ TYPE \ TkBots \ IDENT_LIST \ ACTIONS \ TkEnd$

$| \text{ empty}$

$TYPE \rightarrow TkInt$

$| TkBool$

$| TkCaracter$

$IDEN_LIST \rightarrow IDENT_LIST \ TkComa \ TkIdent$

$ACTIONS \rightarrow ACTIONS \ TkOn \ CONDITION \ TkDosPuntos \ INSTRUCTION_C \ TkEnd$

$| \text{ empty}$

$CONDITION \rightarrow TkActivation$

$| TkDeactivation$

$| TkDefault$

$| EXP_BINARIA$

$INSTRUCTION_C \rightarrow INSTRUCTION_C \ SIMPLE_INSTRUCTION_C$



| SIMPLE_INSTRUCTION_C

INSTRUCTION_E $\rightarrow$ INSTRUCTION_E SIMPLE_INSTRUCTION_E

| SIMPLE_INSTRUCTION_E

SIMPLE_INSTRUCTION_C $\rightarrow$ TkStore EXP_BINARIA TkPunto

| TkCollect TkPunto

| TkCollect TkAs TkIdent TkPunto

| TkDrop EXP_BINARIA TkPunto

| DIRECTION TkPunto

| TkRead TkPunto

| TkRead TkAs TkIdent TkPunto

| TkSend TkPunto

DIRECTION $\rightarrow$ TkLeft

| TkRight

| TkDown

| TkUp

SIMPLE_INSTRUCTION_E $\rightarrow$ TkActivate IDENT_LIST TkPunto

| TkDeactivate IDENT_LIST TkPunto

| TkAdvance IDENT_LIST TkPunto

| TkIf EXP_BINARIA TkDosPuntos INSTRUCTION_E TkEnd



| TkIf EXP_BINARIA TkDosPuntos INSTRUCTION_E TkElse
TkDosPuntos INSTRUCTION_E TkEnd

| TkWhile EXP_BINARIA TkDosPuntos INSTRUCTION_E TkEnd

| CREATE EXECUTE

EXECUTE $\rightarrow$ TkExecute INSTRUCTION_E TkEnd

EXP_BINARIA $\rightarrow$ EXP_BINARIA TkSuma EXP_BINARIA

| EXP_BINARIA TkResta EXP_BINARIA

| EXP_BINARIA TkMult EXP_BINARIA

| EXP_BINARIA TkDiv EXP_BINARIA

| EXP_BINARIA TkMod EXP_BINARIA

| EXP_BINARIA TkConjuncion EXP_BINARIA

| EXP_BINARIA TkDisyuncion EXP_BINARIA

| EXP_BINARIA TkIgual EXP_BINARIA

| EXP_BINARIA TkNoIgual EXP_BINARIA

| EXP_BINARIA TkMenorIgual EXP_BINARIA

| EXP_BINARIA TkMayorIgual EXP_BINARIA

| EXP_BINARIA TkMenor EXP_BINARIA

| EXP_BINARIA TkMayor EXP_BINARIA

EXP_BINARIA $\rightarrow$ TkParAbre EXP_BINARIA TkParCierra

| TkNegacion EXP_BINARIA

| TkResta EXP_BINARIA %prec TkNegacion

```
| TkNum
| TkTrue
| TkFalse
| TkIdent
| TkCharacter
| TkFalse
```

Expresiones

Las expresiones fueron divididas en:

- Expresiones aritméticas.

Se implementaron los siguientes operadores aritméticos:

- $+$: *suma*
- $-$: *resta*
- $*$: *multiplicación*
- $/$: *división*
- $\%$: *módulo*

Estos operadores permiten construir expresiones como:

$3 + 5$

$x * y$

$contador \% 2$

Cada vez que el parser reconoce una expresión aritmética, construye un nodo del tipo BIN_ARITMETICA, almacenando la operación realizada y sus dos operandos.

- Expresiones booleanas.

Para las expresiones booleanas se implementaron los operadores propios del lenguaje BOT:

- $\wedge$: *conjunción lógica (AND)*
- $\vee$: *disyunción lógica (OR)*
- $\sim$: *negación lógica (NOT)*

Las expresiones booleanas generan nodos del tipo BIN_BOOLEANA para operadores binarios y EXP_UNARIA para el operador de negación.

- Expresiones relacionales.

Las expresiones relacionales permiten comparar valores y producir resultados booleanos.

Los operadores implementados fueron:

- $<$: *menor que*
- $\leq$: *menor o igual que*
- $>$: *mayor que*
- $\geq$: *mayor o igual que*
- $=$: *igual que*
- $\neq$: *distinto que*

Cuando una comparación es reconocida, se construye un nodo del tipo BIN_RELACIONAL, que almacena la operación y las expresiones correspondientes a los operandos izquierdo y derecho.

- Expresiones unarias: operador unario de negación.

Manejo de Ambigüedad

La gramática de expresiones es inherentemente ambigua.

Por ejemplo:

$$3 + 4 * 2$$



puede interpretarse de distintas formas dependiendo de la precedencia de los operadores.

Para resolver este problema se definieron niveles de precedencia y asociatividad mediante la estructura precedencia de PLY.

Se otorgó mayor precedencia a:

1. Negación.
2. Multiplicación, división y módulo.
3. Suma y resta.
4. Operadores relacionales.
5. Operadores booleanos.

Esto garantiza que las expresiones sean interpretadas de manera consistente.

Árbol Sintáctico Abstracto

Una vez reconocida una producción válida, el parser construye objetos que representan los distintos nodos del AST.

Se diseñó una jerarquía de clases basada en una clase abstracta denominada node.

A partir de esta clase se derivaron:

- Secuenciación
- Almacenamiento
- Condicional
- RepeticiónIndeterminada
- Binaria
- Unaria
- Valor
- instructions



Cada nodo encapsula únicamente la información semánticamente relevante, descartando elementos puramente sintácticos como palabras clave o delimitadores.

Errores Sintácticos

Se reporta únicamente el primer error sintáctico detectado.

La implementación desarrollada permitió construir un analizador sintáctico funcional para el lenguaje BOT. El sistema es capaz de verificar la validez sintáctica de programas escritos en el lenguaje, construir automáticamente un Árbol Sintáctico Abstracto y generar una representación legible de la estructura reconocida.